Lluvia Estefania Villanueva Lazo. GTID244.

Universidad Tecnológica del Norte de Guanajuato
Organismo Público Descentralizado del Gobierno del Estado
"Educación y progreso para la vida"

Licenciatura en Ingeniería en Tecnologías de la Información e Innovación Digital TSU en Desarrollo de Software Multiplataforma

Asignatura: Estructura de datos

Unidad II.

U2EP1 Ejercicios Tarea Listas - 10%

Profesor: Gabriel Barron Rodríguez

Alumno:

Villanueva Lazo Lluvia Estefanía

1225100383

Lunes 26 de octubre de 2026

Dolores Hidalgo Cuna de la Independencia Nacional, G.T.O

Lluvia Estefania Villanueva Lazo. GTID244.

Introducción

En este documento se presenta la resolución de la serie de ejercicios de la Unidad II sobre la estructura de datos de Listas Enlazadas. El objetivo principal de la práctica fue implementar soluciones óptimas para distintos problemas prácticos (simulación de filas con prioridad, búsqueda de valores mínimos, generación de tickets ordenados y filtrado con ordenamiento alfabético) construyendo nuestras propias estructuras de datos desde cero en Java, sin recurrir a clases nativas avanzadas del lenguaje.

Para lograr un código limpio, modular y eficiente, dividí cada ejercicio en clases independientes (Nodo, la clase correspondiente a la estructura de la lista y Main), utilizando herramientas de lectura rápida como BufferedReader e InputStreamReader para cumplir con las restricciones de tiempo y memoria de los jueces en línea. Asimismo, apliqué un control de versiones riguroso mediante Git y GitHub, trabajando con ramas independientes por cada problema (feature/ejercicio...) y realizando commits granulares por cada archivo desarrollado para garantizar un historial de cambios ordenado y profesional.

Lluvia Estefania Villanueva Lazo. GTID244.

EJERCICIO 1-Formados en la cafetería.

1. **Descripción breve del problema con tus propias palabras.**
   Básicamente se tiene que simular la fila de la cafetería de la escuela, donde llegan alumnos y trabajadores. Solo que los trabajadores tienen prioridad para que los atiendan primero, pero entre los de la misma categoría se respeta el orden de llegada de toda la vida (el primero que llega es al que atienden primero).

2. **Tipo de lista elegida (simple o doble) y la justificación de la elección.**
   Lista Doblemente Enlazada: Me sirvió perfecto porque con los apuntadores al frente y atrás (cabeza y cola) puedo agregar y quitar personas rápidamente sin meterme en líos, cumpliendo con lo que pedía la tarea.

3. **Explicación del algoritmo (o diagrama de nodos y punteros).**
   Lo dividí en tres partes para tener todo ordenado:

- Nodo: Es el que guarda el nombre de la persona y los apuntadores al siguiente y al anterior.
- ListaDoble: Aquí armé los métodos para encolar (agregar al final), desencolar (atender al de adelante) y checar si la fila está vacía.
- Main: Usé un BufferedReader para leer los datos de entrada. Aquí creé dos listas separadas: una para alumnos y otra para trabajadores. Cuando llega el comando de atender, el programa revisa primero si hay trabajadores formados; si hay, atiende a uno de ellos, y si no, se pasa a los alumnos.

4. **Pruebas realizadas, con captura de pantalla de cada una: o Casos de prueba propios (mínimo 3: un caso normal, uno límite como lista vacía o un solo elemento, y uno grande).**

Caso de Prueba 1: Caso Normal (Mezcla de alumnos, trabajadores y atención)

Prueba el funcionamiento intercalando la llegada de alumnos y trabajadores, y verificando que la atención respete la prioridad de los trabajadores.

Lluvia Estefania Villanueva Lazo. GTID244.

```
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio1_FormadosEnLaCafeteria.Main
7
LLEGA ALUMNO luis
LLEGA TRABAJADOR ana
LLEGA ALUMNO jorge
ATIENDE
ATIENDE
LLEGA TRABAJADOR juan
ATIENDE
ana
luis
juan

C:\Listas\Lluvia-Villanueva-listas-java>
```

Caso de Prueba 2: Caso Límite (Llegada de un solo tipo de usuario)

Cuando solo se registran trabajadores, validando que se atiendan en el orden correcto sin que haya alumnos en la fila.

```
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio1_FormadosEnLaCafeteria.Main
3
LLEGA TRABAJADOR carlos
LLEGA TRABAJADOR sofia
ATIENDE
carlos

C:\Listas\Lluvia-Villanueva-listas-java>
```

Caso de Prueba 3: Caso Grande

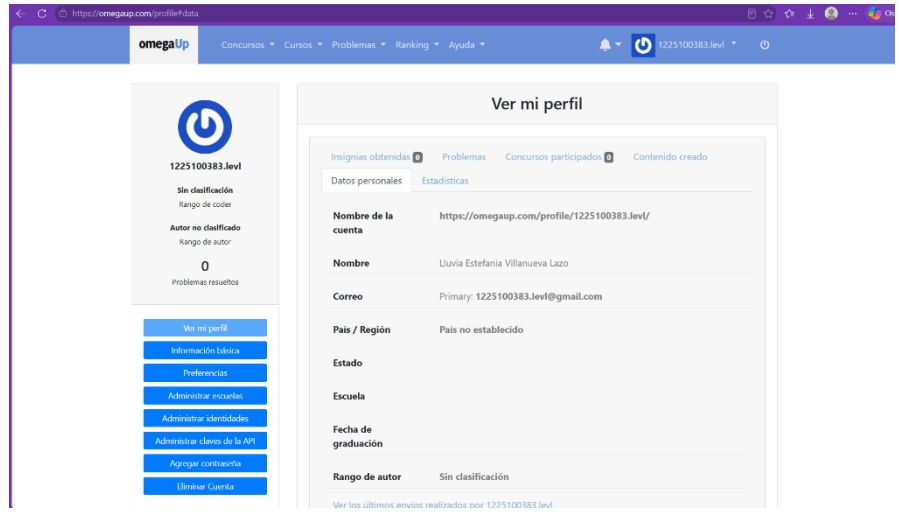
Procesa una secuencia larga.

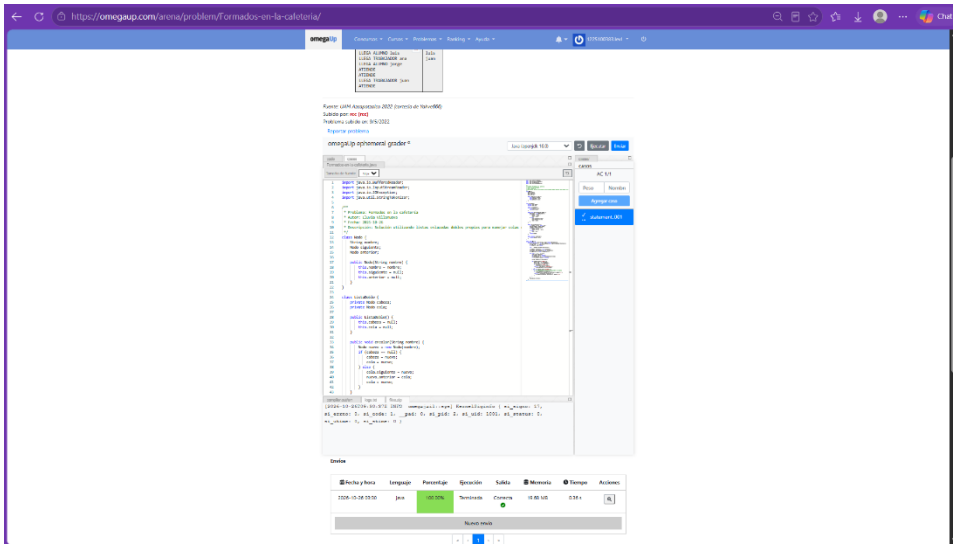
```
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio1_FormadosEnLaCafeteria.Main
10
LLEGA ALUMNO pedro
LLEGA ALUMNO lluvia
LLEGA TRABAJADOR katya
LLEGA ALUMNO fernando
ATIENDE
LLEGA TRABAJADOR marco
ATIENDE
ATIENDE
ATIENDE
ATIENDE
katya
marco
pedro
lluvia
fernando

C:\Listas\Lluvia-Villanueva-listas-java>
```

Captura del veredicto de omegaUp (puntaje obtenido).

Lluvia Estefania Villanueva Lazo. GTID244.





5. Enlace al código correspondiente dentro del repositorio de GitHub.

[Lluvia-Villanueva-listas-java/ejercicio1_FormadosEnLaCafeteria at feature/ejercicio1-cafeteria · Lluvia-18/Lluvia-Villanueva-listas-java]

6. **Conclusiones: dificultades encontradas y cómo se resolvieron.**
   Al principio me trabé un poco acomodando bien los punteros anterior y siguiente (sobre todo para que no me marcara error cuando la lista se quedaba vacía), pero lo fui probando poco a poco. También me ayudó bastante usar BufferedReader. Hacer esto me ayudó a entenderle mucho mejor a cómo funcionan las listas doblemente enlazadas en la vida real.

Lluvia Estefania Villanueva Lazo. GTID244.

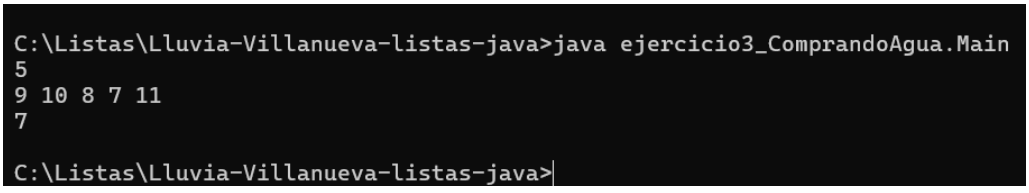
EJERCICIO 3- Comprando agua.

1. **Descripción breve del problema con tus propias palabras.**
   Se trata de ayudar a elegir la mejor opción cuando vas a la tienda. Básicamente te dan un número N de marcas de agua con sus precios, y el programa tiene que revisar todas las opciones para encontrar y mostrar únicamente la botella que tenga el menor costo de todas.

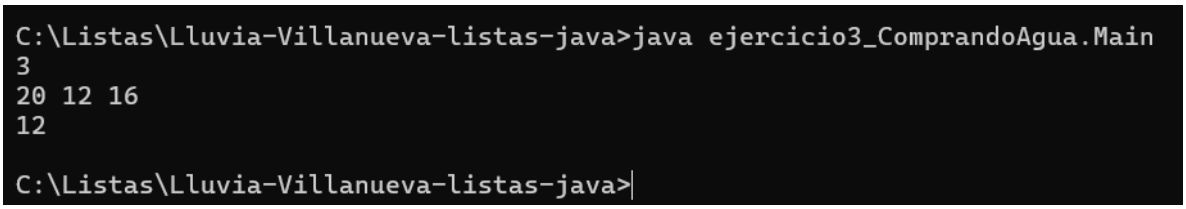
2. **Tipo de lista elegida (simple o doble) y la justificación de la elección.**
   Lista Enlazada (para guardar los precios): Aunque para este problema bastaba con comparar sobre la marcha, usar una lista enlazada me permitió ir almacenando cada precio conforme lo iba leyendo del usuario y recorrerla de manera ordenada mediante nodos.

3. **Explicación del algoritmo (o diagrama de nodos y punteros).**
   Lo separé en tres archivos:
   - Nodo.java: Guarda el precio de la botella y el apuntador al siguiente nodo.
   - ListaPrecios.java: Contiene los métodos para ir agregando cada precio al final de la lista y una función para recorrer los nodos comparándolos para sacar el menor (obtenerMinimo).
   - Main.java: Utilicé un BufferedReader junto con un StringTokenizer para hacer una lectura rápida de los datos. Lee cuántas marcas hay, las mete a la lista y al final imprime el precio más barato.

4. **Pruebas realizadas, con captura de pantalla de cada una: o Casos de prueba propios (mínimo 3: un caso normal, uno límite como lista vacía o un solo elemento, y uno grande).**
   1. **Caso Normal**



   2. Caso Límite

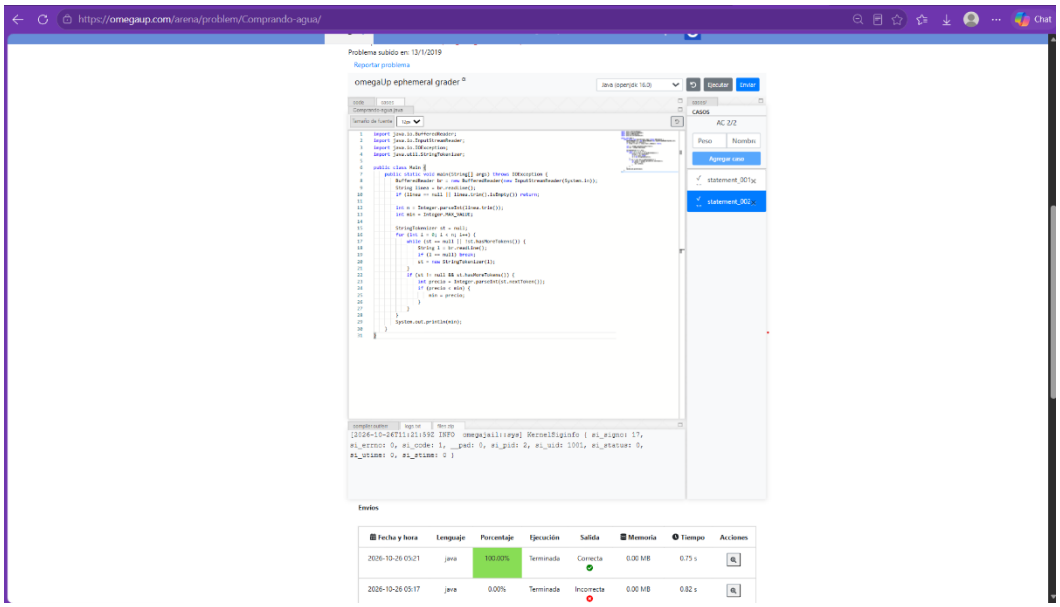


3.Caso Grande

```
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio3_ComprandoAgua.Main
6
50 120 5 99 3 88
3

C:\Listas\Lluvia-Villanueva-listas-java>
```

Captura del veredicto de omegaUp (puntaje obtenido).



5. **Enlace al código correspondiente dentro del repositorio de GitHub.**

   [Lluvia-Villanueva-listas-java/ejercicio3_ComprandoAgua at feature/ejercicio3-comprandoAgua · Lluvia-18/Lluvia-Villanueva-listas-java]

6. **Conclusiones: dificultades encontradas y cómo se resolvieron.**
   Al principio me confundí un poco porque pensé que era una cola con turnos como el ejercicio pasado, pero al leer bien el problema vi que era de buscar mínimos. Tuve que ajustar la lógica para que recorriera la lista comparando el precio actual con el guardado en una variable auxiliar. Hacerlo modular y subirlo con varios commits me ayudó a tener el código limpio y sin enredos con los paquetes.

Lluvia Estefania Villanueva Lazo. GTID244.

EJERCICIO 5- Supermercado

1. **Descripción breve del problema con tus propias palabras.**
   El problema consiste en simular la generación de un ticket de compra en un supermercado. Te dan una cantidad N de productos con sus respectivos nombres y precios, y el programa tiene que organizarlos ordenándolos de mayor a menor precio, imprimiéndolos en forma de lista numerada y mostrando el costo total de toda la compra al final.

2. **Tipo de lista elegida (simple o doble) y la justificación de la elección.**
   Lista Enlazada: Me pareció la mejor opción ya que permite ir agregando cada producto en el orden en que se leía y manipular los nodos para realizar el orden de los precios.

3. **Explicación del algoritmo (o diagrama de nodos y punteros).**
   Lo organicé en tres archivos para mantener un buen diseño:
   - Nodo.java: Almacena la información de cada artículo (su nombre, su precio y el enlace al siguiente nodo).
   - ListaProductos.java: Contiene el método para insertar nuevos productos al final de la lista y un algoritmo de ordenamiento para comparar los precios de los nodos y reacomodarlos de menor a menor.
   - Main.java: Utilicé BufferedReader y StringTokenizer para agilizar la lectura de los datos de entrada, calculé la suma total de los precios, mandé a ordenar la lista y finalmente construí la salida en formato de ticket.

4. **Pruebas realizadas, con captura de pantalla de cada una: o Casos de prueba propios (mínimo 3: un caso normal, uno límite como lista vacía o un solo elemento, y uno grande).**

Lluvia Estefania Villanueva Lazo. GTID244.

1.Caso Normal

```
C:\Listas\Lluvia-Villanueva-listas-java>javac ejercicio5_Supermercado/*.java

C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio5_Supermercado.Main
8
axion 23.68
fud 23.00
LaLa 23.01
Petalo 129.5
2kg_manzanas 58.6
BimboG 30.5
MayonesaM 50.5
Colgate 23
1 | Petalo | 129.5
2 | 2kg_manzanas | 58.6
3 | MayonesaM | 50.5
4 | BimboG | 30.5
5 | axion | 23.68
6 | LaLa | 23.01
7 | fud | 23.0
8 | Colgate | 23.0
361.79
C:\Listas\Lluvia-Villanueva-listas-java>
```

2. Caso Límite (Pocos productos)

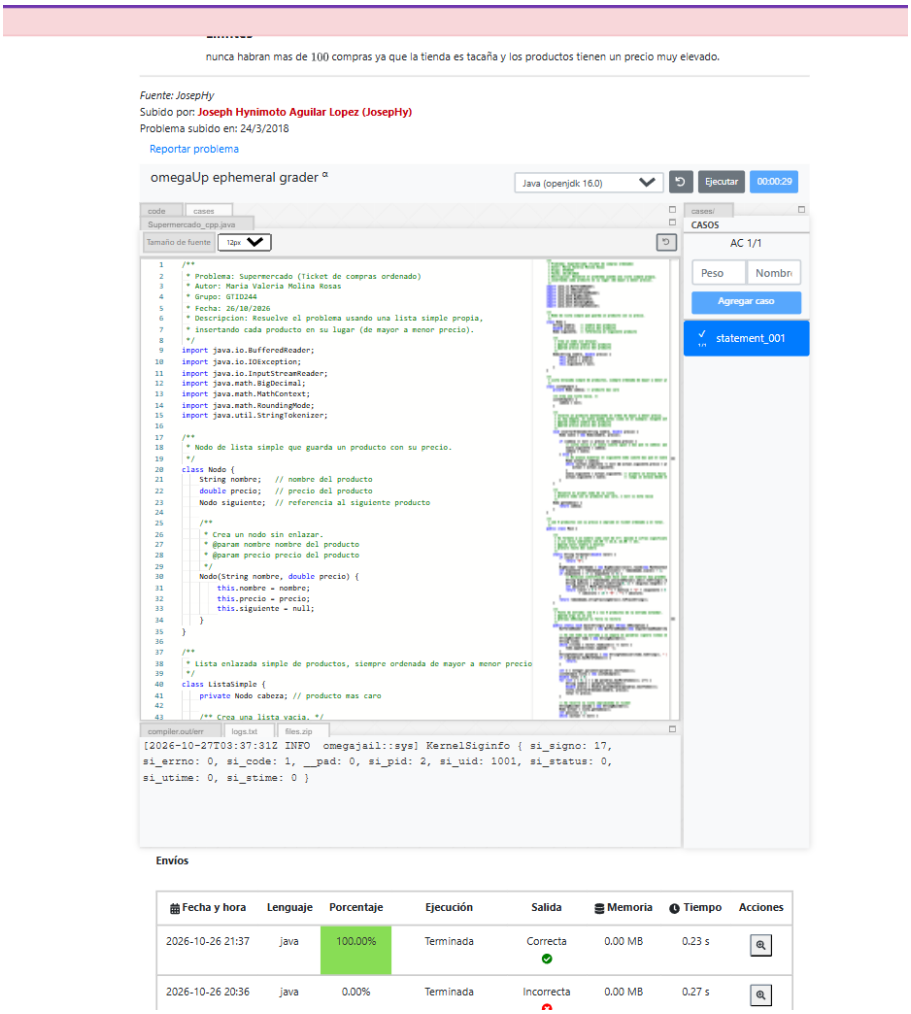
```
361.79
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio5_Supermercado.Main
2
Leche 25.50
Pan 42.00
1 | Pan | 42.0
2 | Leche | 25.5
67.5
C:\Listas\Lluvia-Villanueva-listas-java>
```

3.Caso Grande (Varios productos y precios variados)

```
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio5_Supermercado.Main
5
Arroz 19.90
Jabon 35.00
Aceite 48.50
Sal 15.00
Atun 20.00
1 | Aceite | 48.5
2 | Jabon | 35.0
3 | Atun | 20.0
4 | Arroz | 19.9
5 | Sal | 15.0
138.4
C:\Listas\Lluvia-Villanueva-listas-java>
```

Captura del veredicto de omegaUp (puntaje obtenido).

Lluvia Estefania Villanueva Lazo. GTID244.



5. **Enlace al código correspondiente dentro del repositorio de GitHub.**
   [Lluvia-Villanueva-listas-java/ejercicio5_Supermercado at feature/ejercicio5-supermercado · Lluvia-18/Lluvia-Villanueva-listas-java]

6. **Conclusiones: dificultades encontradas y cómo se resolvieron.**
   Este ejercicio fue el más desafiante de la práctica porque implicó combinar la creación de listas enlazadas con un algoritmo de ordenamiento sobre los nodos. Al principio lidiar con los decimales y el orden exacto del ticket requirió de paciencia, pero separar la lógica en clases independientes ayudó a estructurar el código de forma limpia. El uso de ramas y commits en Git hizo que todo el proceso de desarrollo fuera muy ordenado y fácil de respaldar.

Lluvia Estefania Villanueva Lazo. GTID244.

EJERCICIO 7- Cuadro de Honor

1. **Descripción breve del problema con tus propias palabras.**
   El problema consiste en evaluar a un conjunto de calificaciones de alumnos, para filtrar únicamente a aquellos que obtuvieron un promedio de (9.5 o más). Una vez seleccionados, el programa debe organizarlos y mostrarlos ordenados alfabéticamente por su nombre.
2. **Tipo de lista elegida (simple o doble) y la justificación de la elección.**
   Lista Enlazada: Me permitió almacenar dinámicamente los registros de cada estudiante conforme se leían de la entrada y aplicar las reglas de filtrado y ordenamiento.
3. **Explicación del algoritmo (o diagrama de nodos y punteros).**
   Lo organicé en tres archivos:
   - Nodo.java: Almacena de manera individual el nombre completo del alumno (incluyendo espacios), su calificación numérica y el apuntador al siguiente nodo.
   - ListaAlumnos.java: Contiene los métodos para insertar nuevos alumnos en la estructura y el algoritmo de ordenamiento alfabético.
   - Main.java: Utiliza BufferedReader para una lectura eficiente, valida la condición de excelencia (≥ 9.5), descarta a los promedios menores y construye la salida final.

4. **Pruebas realizadas, con captura de pantalla de cada una: o Casos de prueba propios (mínimo 3: un caso normal, uno límite como lista vacía o un solo elemento, y uno grande).**

   1.Caso Normal



   2.Caso Límite (Alumnos justo en la raya del 9.5)

```
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio7_CuadroDeHonor.Main
2
Ana Lopez
9.4
Luis Perez
9.8
Luis Perez

C:\Listas\Lluvia-Villanueva-listas-java>
```

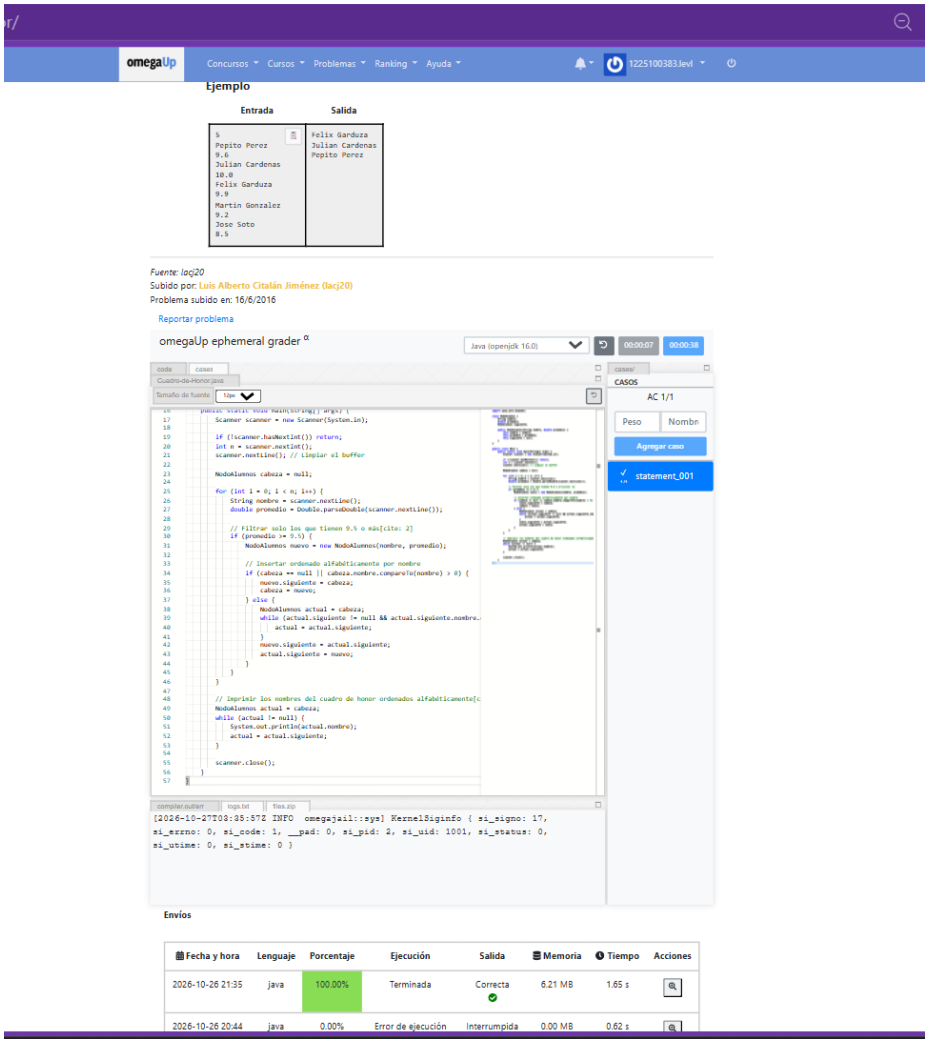
3.Caso Grande

```
C:\Listas\Lluvia-Villanueva-listas-java>java ejercicio7_CuadroDeHonor.Main
4
Zoe Gomez
9.8
Carlos Ruiz
9.5
Alan Smith
9.9
Beto Perez
9.0
Alan Smith
Carlos Ruiz
Zoe Gomez

C:\Listas\Lluvia-Villanueva-listas-java>
```

Lluvia Estefania Villanueva Lazo. GTID244.

Captura del veredicto de omegaUp (puntaje obtenido).



5. **Enlace al código correspondiente dentro del repositorio de GitHub.**
   [Lluvia-Villanueva-listas-java/ejercicio7_CuadroDeHonor at feature/ejercicio7-cuadroDeHonor · Lluvia-18/Lluvia-Villanueva-listas-java]

7. **Conclusiones: dificultades encontradas y cómo se resolvieron.**
   Este ejercicio me sirvió al lidiar con entradas compuestas por varias líneas para un solo elemento (nombre y calificación por separado) y aplicar filtros condicionales estrictos. El manejo de ramas y commits en Git hizo que el desarrollo fuera más limpio.

Lluvia Estefania Villanueva Lazo. GTID244.

Conclusión General

En lo personal, esta práctica me ayudó muchísimo a comprender a fondo el funcionamiento interno de las listas enlazadas (tanto simples como dobles) y a perderle el miedo a la manipulación manual de nodos y apuntadores. Trabajar sin librerías nativas me obligó a razonar detalladamente cómo enlazar, recorrer, filtrar y reacomodar los elementos en memoria sin romper las referencias o causar errores cuando la lista queda vacía o con un solo nodo.

Además, optimizar la lectura de datos me enseñó la importancia de la eficiencia en el procesamiento de entradas y salidas para evitar errores de tiempo de ejecución en plataformas como omegaUp. Integrar estas soluciones con un flujo de trabajo profesional en Git (usando ramas separadas y mensajes de commit descriptivos) hizo que el desarrollo fuera mucho más estructurado, limpio y fácil de respaldar. En resumen, fue una práctica muy completa que reforzó mis habilidades en lógica , algoritmos sobre estructuras dinámicas y control de versiones.